

# Kuber: Geometry-General Neural Surrogates for Conjugate Heat Transfer

Shubh Jain\*     Hardik Agarwal\*  
github.com/ShubhJain007/Kuber

\*Equal contribution

**Abstract**—Conjugate heat transfer (CHT) — the coupled transport of heat through a solid conductor and a surrounding moving fluid — governs the thermal design of heatsinks, cold plates, heat exchangers, power electronics, and battery packs. High-fidelity computational fluid dynamics (CFD) resolves these fields but costs minutes to hours per geometry, prohibitive inside a design loop. We present Kuber, an open framework for building neural surrogates of CHT, and SurfaceGeoTransolver, a geometry-general model that predicts the full steady field ( $U$ ,  $T$ ,  $p$ ) at an arbitrary query point cloud in a sub-second. The model couples a physics-attention transformer core with a surface-geometry encoder, so it ingests raw boundary geometry (a surface point cloud with normals) and requires no analytic signed-distance field, and it is conditioned on the full set of physical scalars plus a device-class flag. On the public SIMSHIFT heatsink benchmark it attains 12.14 K temperature RMSE on the medium out-of-distribution split, improving on the best previously published result (12.41 K) while using no unsupervised domain adaptation, which every reported baseline relies on. A single set of weights spans two physically distinct device classes — buoyancy-driven heatsinks in air and forced-convection liquid cold plates — and pretraining on a self-generated OpenFOAM corpus lowers out-of-distribution error further. We additionally verify numerical soundness: the predicted temperature gradient never exceeds the physical gradient at fin tips and corners (explosion fraction zero, no NaN/Inf). All results are reproducible with the released code and data.

**Index Terms**—conjugate heat transfer, neural operators, surrogate modeling, transformers, physics-informed machine learning, out-of-distribution generalization.

## I. INTRODUCTION

Thermal design is an inner loop. An engineer proposes a geometry — a fin array, a cold-plate channel layout — and must know the resulting temperature and flow field before iterating. The field is produced by solving the coupled Navier–Stokes and energy equations across a solid and a fluid region: *conjugate heat transfer* (CHT). Finite-volume CFD solves this accurately but slowly; in our measurements a single steady case takes a median of 22 minutes and up to nearly two hours for fine geometries. Sweeping thousands of designs, or closing an optimization loop, is therefore dominated by solver cost.

Neural surrogates promise to amortize this cost: learn the geometry-to-field map once, then evaluate it in milliseconds. Recent operator-learning and transformer methods have made this practical for external aerodynamics, but electronics-cooling CHT has received far less attention, has no widely used open benchmark harness, and no shared, license-clean data-generation recipe. Two requirements make the problem hard. First, *geometry generality*: a useful surrogate must accept arbitrary CAD, which rules out geometry encodings that presuppose an analytic shape. Second, *generalization*: the surrogate must predict geometries it never saw in training.

We make three contributions. (i) SurfaceGeoTransolver, a surface-conditioned physics transformer that predicts the full five-channel CHT field at any query point from raw boundary geometry and physical conditioning, reaching state of the art on a public benchmark without domain adaptation. (ii) A reproducible data engine generating a self-labeled OpenFOAM CHT corpus across fluids, regimes, shapes, and two device classes, with a convergence gate and a verified mesh-convergence protocol. (iii) An honest evaluation harness — per-field normalized error, near-wall fidelity, a numerical no-explosion proof, and a value-of-data ablation — with released code, data, machine-readable results, and an interactive viewer.

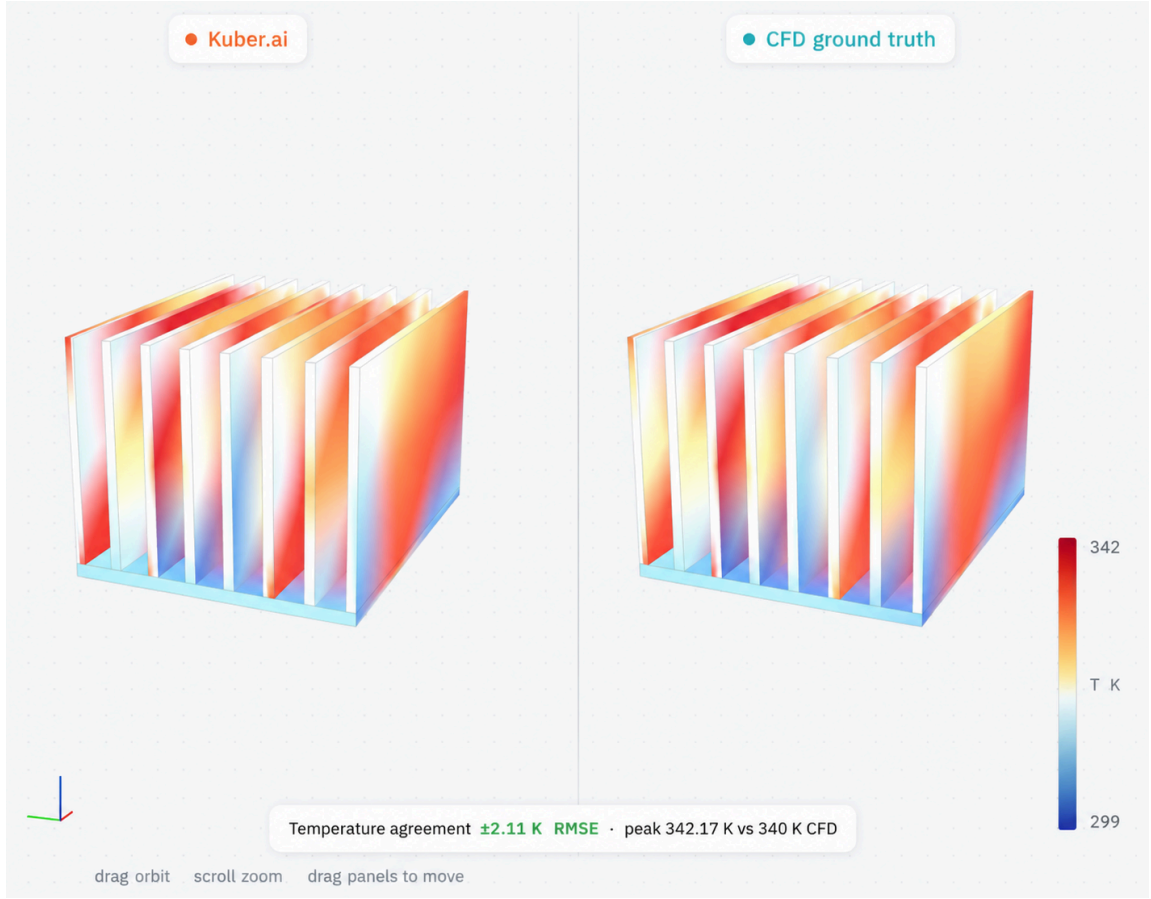


Fig. 1. Kuber prediction vs. CFD ground truth for a heatsink simulation: surface temperature field on the finned solid,  $\pm 2.11$  K RMSE agreement (peak 342.17 vs. 340 K CFD), computed 7,000 $\times$  faster than the CFD solver.

## II. RELATED WORK

*Neural operators.* DeepONet [7] and the Fourier Neural Operator [6] established learning maps between function spaces; GINO [8] extended operator learning to large 3D geometries. These are strong on fixed domains but do not natively consume arbitrary boundary geometry with per-node queries. *Transformer PDE (partial-differential-equation) solvers.* Transolver [3] introduced physics attention over learned slices; UPT [4] and the anchored/branched variant (AB-UPT) [5] scale transformer surrogates to industrial CFD. Our core builds on the GeoTransolver implementation in NVIDIA PhysicsNeMo [10]; our contribution is the surface-conditioning branch and the CHT-specific conditioning and training recipe. *Geometry representations.* Signed-distance fields (SDFs) are data-efficient when an analytic shape exists, but none exists for arbitrary CAD (computer-aided-design geometry); following the surface branch of AB-UPT [5] we encode the boundary directly as a point cloud with normals. *Spectral fidelity.* One-shot regression is spectrally biased; PDE-Refiner [9] restores high-frequency content, which we adopt as an optional head. *Benchmarks.* SIMSHIFT [2] is a public benchmark for distribution shift in engineering simulation; we use its heatsink task as our primary evaluation.

## III. PROBLEM FORMULATION

A CHT case consists of a solid conductor with boundary  $\Gamma$  immersed in a fluid domain  $\Omega$ . Given the boundary geometry

and operating conditions, we seek the steady field at any query point  $x \in \Omega$ :

$$f_{\theta} : (x, \Gamma, c) \mapsto (U_x, U_y, U_z, T, p_{\text{rgh}}), \quad (1)$$

where  $U$  is velocity,  $T$  temperature, and  $p_{\text{rgh}} = p - \rho gh$  the reduced pressure. The conditioning vector  $c$  collects fluid properties (Prandtl number, density, specific heat, dynamic viscosity), boundary conditions (BCs; a wall temperature for heatsinks or a heat flux for cold plates, and the ambient/inlet temperature), inlet velocity, and a binary device-class flag. The boundary  $\Gamma$  is presented as surface points with outward unit normals. Nothing in the input assumes a parametric shape family.

## IV. METHOD

### A. SurfaceGeoTransolver

A *surface encoder* maps the boundary point cloud with normals to permutation-invariant geometry tokens via a shallow self-attention stack. A *local cross-attention* module produces, for every query node, a geometry descriptor by attending over its  $k = 16$  nearest surface tokens, so each query carries a local view of the nearby boundary. This descriptor is concatenated with the broadcast conditioning vector to form the per-node embedding. The *core* is a GeoTransolver physics-attention transformer (256 hidden units, 12 layers, 8 heads, 64 physics slices, with multiscale local ball-query features) that regresses the five z-normalized (standardized: zero mean, unit variance) targets. The

model has approximately 14.3 M parameters. An optional PDE-Refiner head can replace one-shot regression (detailed in Sec. IV-B). Geometry may alternatively be supplied as a (directional) signed-distance field for parametric families, but the surface-encoder configuration is the one we report because it is the only one applicable to arbitrary CAD.

Formally, write the surface as points  $p_i \in \mathbb{R}^3$  with unit normals  $n_i$  ( $i = 1 \dots N_s$ ), the query cloud as  $x_j$  ( $j = 1 \dots N$ ), and the conditioning as  $c \in \mathbb{R}^C$ . The *surface encoder* embeds each element with a multi-layer perceptron (MLP) and applies an  $L_s$ -layer pre-norm Transformer of multi-head self-attention (MHSA), feed-forward networks (FFN), and layer normalization (LN):

$$h_i^0 = \text{MLP}([p_i; n_i]), \quad H \leftarrow H + \text{MHSA}(\text{LN } H), \quad H \leftarrow H + \text{FFN}(\text{LN } H), \quad \text{MHSA}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d})V, \quad (2)$$

yielding geometry tokens  $z_i = H_i^{(L_s)}$ . For each query  $x_j$ , let  $N_k(j)$  be its  $k$  nearest surface points (k-nearest neighbors, kNN); with relative positions  $r_{ji} = x_j - p_i$ , learned bias  $b_{ji} = \text{MLP}_r(r_{ji})$ , and query  $q_j = \text{MLP}_q(x_j)$ , the *local geometry descriptor* is

$$\alpha_{ji} = \text{softmax}_{i \in N_k(j)} (q_j^\top (W_K z_i + b_{ji}) / \sqrt{d_h}), \quad g_j = W_O \sum_i \alpha_{ji} (W_V z_i + b_{ji}) \quad (3)$$

(multi-head; heads omitted). Locality and relative positions make  $g_j$  compositional — a 10-fin heatsink is locally “more of the same” fin-gap patches seen at 5–8 fins. The per-node

embedding  $e_j = [g_j; c]$  is lifted,  $u_j^0 = \text{MLP}([e_j; x_j])$ , and passed through  $L_c$  *physics-attention* blocks. Each block softly assigns the  $N$  nodes to  $M$  learned slices, aggregates them into slice tokens, attends among the  $M$  tokens, and scatters back:

$$w_{jm} = \text{softmax}_m(u_j^\top \theta_m), \quad t_m = (\sum_j w_{jm} u_j) / (\sum_j w_{jm}), \quad u_j \leftarrow u_j + \sum_m w_{jm} \text{MHSA}(T)_m \quad (4)$$

followed by a multiscale ball-query aggregation of neighbors at radii  $\{0.05, 0.25\}$  and an FFN, each with a residual and LN. The slice mechanism costs  $O(NM)$  with  $M \ll N$  instead of the  $O(N^2)$  of dense attention. A linear head gives the z-normalized field, trained by mean-squared error and de-normalized to physical units:

$$\hat{y}_j = W_o u_j^{(L_c)} \in \mathbb{R}^5, \quad \mathcal{L}(\theta) = (1/BN) \sum_{b,j} \|\hat{y}_{bj} - y_{bj}\|^2, \quad \text{field} = \hat{y}_j \odot \sigma_y + \mu_y. \quad (5)$$

Here  $N_s$  and  $N$  are the numbers of surface and query points and  $C$  the number of conditioning scalars;  $L_s$ ,  $L_c$  the encoder and core depths and  $M$  the slice count;  $d$ ,  $d_h$  the model and per-head attention widths and  $d_s$ ,  $d_g$  the token and descriptor widths;  $Q$ ,  $K$ ,  $V$  the attention query/key/value matrices;  $W_K$ ,  $W_V$ ,  $W_O$ ,  $W_o$  learned projection matrices and  $\theta_m$  the  $M$  learned slice queries;  $\sigma_y$ ,  $\mu_y$  the per-channel target standard deviation and mean,  $\odot$  the elementwise product,  $B$  the batch size, and  $\theta$  all trainable parameters. Every MLP is two-layer with a GELU activation.

## SurfaceGeoTransolver

Geometry-general conjugate-heat-transfer surrogate — internal layers shown

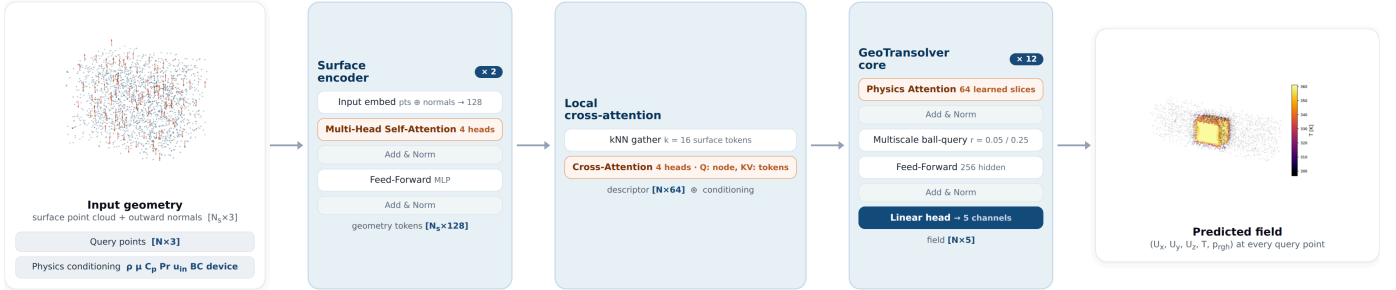


Fig. 2. SurfaceGeoTransolver. Three inputs — the boundary surface point cloud with normals, the query points, and the physics conditioning — feed a neural spine (surface encoder → local kNN cross-attention → concatenated per-node embedding → GeoTransolver physics-attention core) that predicts the five-channel field at every query node.

### B. PDE-Refiner Head (Optional)

*Why we use it.* One-shot regression under a mean-squared-error loss is *spectrally biased*: low-amplitude high-frequency modes — sharp fin-tip gradients, thin thermal boundary layers — contribute little to the squared error, so a single forward pass tends to over-smooth them. PDE-Refiner [9] removes this bias by casting field prediction as an iterative *denoising* process, whose training objective forces the network to represent structure at every spatial frequency; as a by-product, sampling the noise yields an ensemble and hence an uncertainty estimate.

*How we use it.* The same network  $f_\theta$  serves as both the signal predictor and the denoiser, conditioned on a refinement step  $k \in \{0, \dots, K\}$  through a sinusoidal step embedding and an extra noised-field input channel. With a geometric noise schedule  $\sigma_k = \sigma_{\min}^{k/K}$  ( $\sigma_0 = 0$ ), each training sample draws  $k \sim$

$\text{Uniform}\{0, \dots, K\}$ : at  $k = 0$  the network regresses the field  $y$  from the conditioning; at  $k \geq 1$  it is given the target corrupted by noise,  $y + \sigma_k \epsilon$  ( $\epsilon \sim \mathcal{N}(0, I)$ ), and regresses the noise  $\epsilon$ . At inference we take the initial one-shot prediction and apply  $K$  refinement steps,  $u \leftarrow f_\theta(\cdot, 0); \quad k = 1 \dots K: \quad u \leftarrow (u + \sigma_k \epsilon) - \sigma_k f_\theta(u + \sigma_k \epsilon, k), \quad (6)$

so each step subtracts the network's predicted noise and restores fine structure; drawing several noise seeds  $\epsilon$  gives an ensemble whose spread is a calibrated uncertainty. All results in this paper use the one-shot model ( $k = 0$  only); the refiner is a drop-in head for applications that need spectral fidelity or uncertainty, and seeds the Uncertainty pillar of the suite.

### C. Data Engine

We generate CHT data with OpenFOAM's steady buoyant solver (buoyantSimpleFoam) over a single fluid region, treating

the solid as a heated wall (heatsinks) or heat-flux surface (cold plates). A parametric generator samples geometry and conditions; the geometry is meshed with blockMesh and snappyHexMesh plus prism layers, solved, and exported to 16,384 fluid nodes with the five field channels, the boundary surface cloud with normals, and a JSON of conditions. The pipeline is resumable and gated: a case is written only after a converged solve, and a physics filter rejects unphysical results. Sampling is seeded, so the corpus regenerates deterministically. It is entirely self-generated (Table I).

TABLE I  
DATA-ENGINE COVERAGE OF THE SELF-GENERATED CORPUS

| Axis     | Coverage                                              |
|----------|-------------------------------------------------------|
| Fluids   | air, water, oil (Pr $\approx$ 292), glycol            |
| Regimes  | natural + forced convection                           |
| Shapes   | fins, plate, cube, pin-fin; duct channels             |
| Devices  | heatsink (wall- $T$ ), cold plate (heat-flux)         |
| Fidelity | $\approx$ 1.4 M cells/case $\rightarrow$ 16,384 nodes |

## V. EXPERIMENTAL SETUP

**Benchmark.** Our primary, externally comparable evaluation is the SIMSHIFT [2] heatsink task at *medium* difficulty: train on fin counts 5–8, test on the shifted, out-of-distribution (OOD) target of fin counts 10–12, which never appear in training. We train from scratch on the 222 training cases; normalizers are fit on the source training split only. **Metrics.** We report per-field root-mean-square error (RMSE) in physical units and normalized RMSE (nRMSE); the benchmark’s primary selector is mean nRMSE across the five fields. We additionally report near-wall  $T$ -RMSE and edge gradient ratios (Sec. VI-D). No baseline uses *unsupervised domain adaptation* (UDA) — training-time adaptation to the *unlabeled* target (test) distribution, e.g. aligning source- and target-domain feature statistics to shrink the out-of-distribution gap — in our runs; the published baselines do, which is their best case. **Training.** Adam [11] at learning rate  $10^{-3}$ , batch size 4 cases (16,384 query nodes each), with a warmup–stable–decay schedule: a 5-epoch linear warmup, a stable phase held until the validation mean-nRMSE plateaus (patience 25 epochs), then a 30-epoch cosine decay to  $10^{-6}$ ; the best-validation checkpoint is kept. Targets are z-scored per channel and coordinates mapped to a per-case unit frame. The encoder uses  $d_s=128$  (2 layers, 4 heads); the descriptor  $d_g=64$  with  $k=16$ ; the core is 256-wide, 12 layers, 8 heads,  $M=64$  slices, ball-query radii  $\{0.05, 0.25\}$  with  $\{8, 32\}$  neighbors

( $\approx$ 14.3 M parameters). Baseline numbers are quoted from the SIMSHIFT paper.

## VI. RESULTS

### A. SIMSHIFT Heatsink Benchmark

On temperature — the design-critical field — SurfaceGeoTransolver improves on the best previously published result while using no UDA (Table II). The paper reports only temperature and velocity RMSE on the target domain; baseline parameter counts are not published but their released configurations are comparably sized ( $\sim$ 10–15 M), so the improvement is not from scale.

TABLE II  
SIMSHIFT HEATSINK, MEDIUM (OOD) SPLIT. BASELINES INCLUDE UDA; OURS DOES NOT. LOWER IS BETTER.

| Model                       | UDA      | Temp RMSE (K) | Vel. RMSE (m/s) |
|-----------------------------|----------|---------------|-----------------|
| <b>SurfaceGeoTransolver</b> | <b>X</b> | <b>12.14</b>  | 0.044           |
| UPT (prev. best) [4]        | ✓        | 12.41         | 0.039           |
| Transolver [3]              | ✓        | 13.43         | 0.041           |
| PointNet [1]                | ✓        | 17.43         | 0.044           |

The benchmark number is itself out of distribution. Table III separates in-distribution accuracy (4.29 K) from the shifted target (12.14 K zero-shot on unseen fin counts).

TABLE III  
SOURCE (IN-DISTRIBUTION) VS. TARGET (OOD) DETAIL FOR OUR MODEL

| Domain            | $T$ (K) | Vel.  | $P_{rgh}$ | mean nRMSE |
|-------------------|---------|-------|-----------|------------|
| source (in-dist.) | 4.29    | 0.025 | 203       | 0.287      |
| target (OOD)      | 12.14   | 0.044 | 2303      | 0.671      |

### B. One Model, Two Device Classes

A single set of weights, distinguished only by the device flag and fluid/BC conditioning, spans a wall-temperature, buoyancy-driven heatsink in air and a heat-flux, forced-liquid cold plate. Evaluated per class on held-out cases from our corpus (there is no public cold-plate CHT benchmark), the model reaches the errors in Table IV. The cold-plate mean nRMSE (0.028) is close to the in-distribution value. These numbers are on our own corpus and are not comparable to the SIMSHIFT numbers.

TABLE IV  
MULTI-GEOMETRY MODEL, HELD OUT PER DEVICE CLASS (OUR CORPUS)

| Held-out class         | $T$ RMSE (K) | $T$ nRMSE | mean nRMSE |
|------------------------|--------------|-----------|------------|
| cold plates            | 3.11         | 0.039     | 0.028      |
| heatsinks              | 5.13         | 0.065     | 0.145      |
| in-distribution (both) | 1.72         | 0.022     | 0.027      |

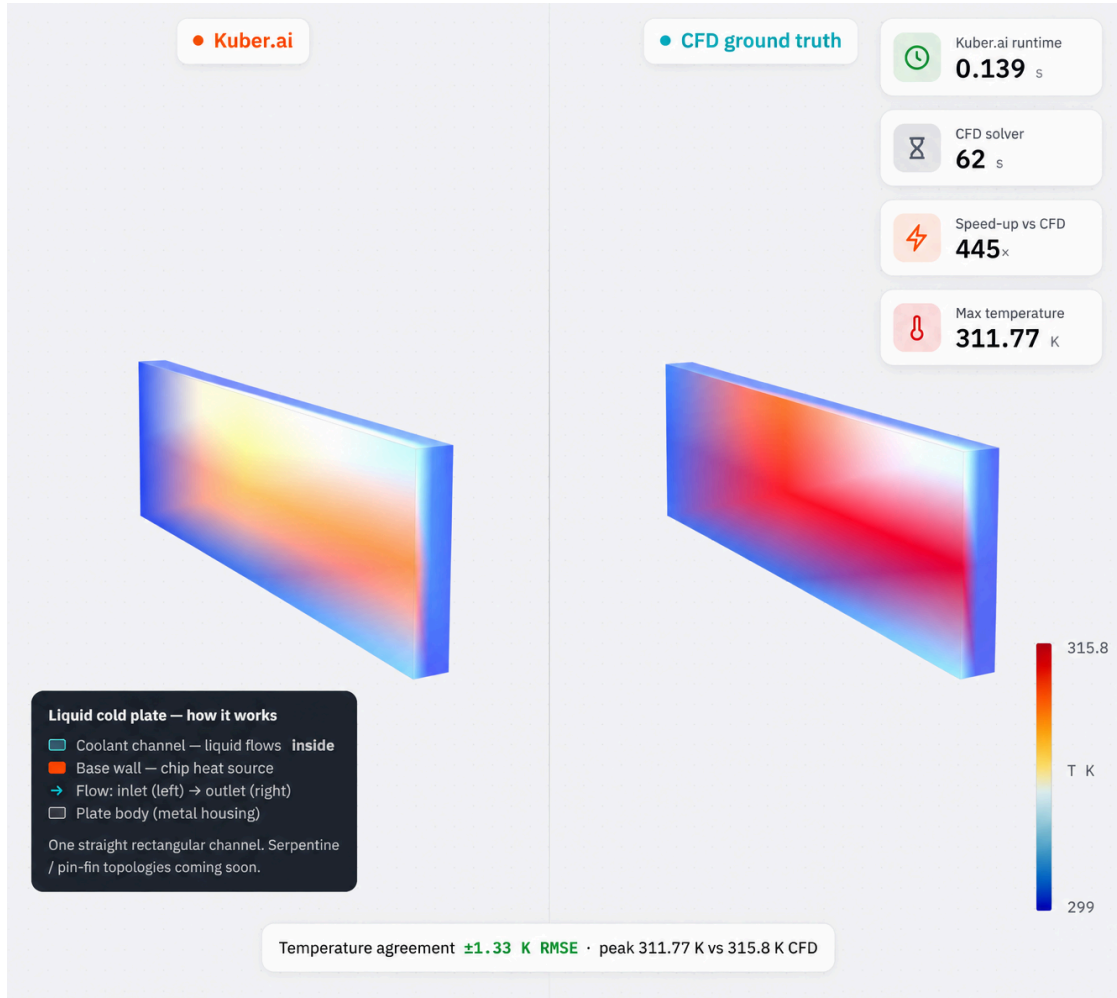


Fig. 3. Kuber prediction vs. CFD ground truth for a liquid cold-plate simulation: surface temperature field,  $\pm 1.33$  K RMSE agreement (peak 311.77 vs. 315.8 K CFD), computed 445 $\times$  faster than the CFD solver.

### C. Value of the Self-Generated Corpus

Does pretraining on the corpus help beyond the benchmark's own training set? A controlled A/B with the same model and SIMSHIFT fine-tuning, changing only the initialization (Table V). Pretraining helps materially at the easy shift ( $\sim 19\%$ ) and in-distribution ( $\sim 12\%$ ), modestly at medium, and is neutral at the hardest shift, which the corpus does not yet cover. The only changed ingredient is the corpus.

TABLE V  
TEMPERATURE RMSE (K); FROM SCRATCH VS. CORPUS-PRETRAINED THEN FINE-TUNED

| Shift    | from scratch | pretrained   | $\Delta$ |
|----------|--------------|--------------|----------|
| easy     | 8.99         | <b>7.28</b>  | $-19\%$  |
| medium   | 12.94        | <b>12.38</b> | $-4.3\%$ |
| hard     | 14.42        | 14.43        | $\pm 0$  |
| in-dist. | 4.63         | <b>4.09</b>  | $-12\%$  |

**Why it helps — and why it is not leakage.** With only a few hundred fine-tuning cases, initialization matters: pretraining on the broader OpenFOAM corpus supplies a physics prior — the coupling of temperature and flow, near-wall behavior, and boundary-layer structure — that is hard to learn from the small target set alone, which is why the gain is largest at the easy shift and in-distribution and tapers at the hardest shift the corpus does not cover. Critically, **no held-out evaluation data is ever seen**

**during pretraining or fine-tuning:** the pretraining corpus is our own OpenFOAM data, entirely disjoint from the SIMSHIFT benchmark and its splits; the SIMSHIFT target (out-of-distribution) split is used only for evaluation; and in the multi-geometry setting the held-out per-device cases are excluded from training. The gains therefore reflect transfer of physics, not exposure to the test distribution.

### D. Numerical Stability

A deployable surrogate must reproduce steep near-wall gradients without over-smoothing or emitting unphysical spikes. We measure the ratio of predicted to CFD temperature-gradient magnitude in the edge band and at the steepest peaks (Table VI). Every ratio is at or below unity, in and out of distribution — the surrogate never produces a steeper-than-physical gradient — with an explosion fraction of exactly zero and no NaN/Inf. It is not merely over-smoothing: the in-distribution edge ratio ( $0.97 \approx 1.0$ ) reproduces the true edge structure, while out of distribution it errs slightly conservative, the safe direction.

TABLE VI  
PREDICTED/CFD TEMPERATURE-GRADIENT RATIO AT EDGES (1.0 = FAITHFUL)

| Metric                      | in-dist. | OOD   |
|-----------------------------|----------|-------|
| edge-band $\nabla T$ ratio  | 0.969    | 0.746 |
| steepest-peak (p99.9) ratio | 0.938    | 0.725 |
| explosion fraction          | 0        | 0     |
| NaN / Inf count             | 0        | 0     |

### E. Speed and Data Fidelity

Inference is a single forward pass whose cost is independent of geometry, unlike CFD whose cost grows with mesh size. Against measured OpenFOAM solve times (median 22 minutes, up to 117 for fine geometries), a sub-second forward pass is up to four orders of magnitude ( $\approx 10,000\times$ ) faster; we report the inference figure as an estimate pending exact per-GPU timing. On fidelity, three prism boundary layers recover the near-wall hot spot to within 0.1 K of a fine mesh (378.8 vs. 378.9 K) at  $\approx 2.7\times$  fewer cells.

### F. Ablations

Table VII isolates the main design choices on the SIMSHIFT medium split. The surface encoder gives the best temperature and, unlike the analytic SDF, applies to arbitrary CAD; the SDF variant is data-efficient (lower averaged nRMSE, 0.593, and velocity, 0.038 m/s) but presupposes an analytic shape. Full 12-scalar physics conditioning improves temperature by 0.8 K over wall-temperature-only, confirming the fluid-property and boundary-condition scalars carry signal. Corpus pretraining lowers the reduced-conditioning model from 12.94 to 12.38 K (Sec. VI-C); both the full-conditioning and the pretrained models pass UPT's 12.41 K. Separately, feeding the surface as a per-node descriptor (used here) generalized better than AB-UPT-style per-block cross-attention on the raw surface cloud at this data scale.

TABLE VII  
ABLATIONS ON SIMSHIFT MEDIUM (OOD). TEMPERATURE RMSE (K); LOWER IS BETTER.

| Factor                         | Setting                    | Temp RMSE (K) |
|--------------------------------|----------------------------|---------------|
| geometry input                 | surface encoder (ours)     | <b>12.14</b>  |
|                                | analytic SDF               | 13.07         |
| conditioning                   | full, 12 scalars (ours)    | <b>12.14</b>  |
|                                | reduced, wall- $T$ only    | 12.94         |
| pretraining<br>(reduced cond.) | reduced, from scratch      | 12.94         |
|                                | reduced, corpus-pretrained | <b>12.38</b>  |

## VII. DISCUSSION AND LIMITATIONS

We state the boundaries plainly. The corpus is air-dominated (tens of oil/glycol cases), so the model is strongest on air. On

SIMSHIFT the OOD axis is fin count alone; leave-one-fluid-out and leave-one-shape-out remain future work. Cold plates are currently straight-channel, so the multi-geometry result shows device-class generality rather than topology extrapolation. On the averaged-nRMSE selector our temperature lead is bought with slightly higher velocity and pressure error, reported transparently. Inference latency is an estimate. The multi-geometry numbers are on our own corpus and are not comparable to the public benchmark.

## VIII. CONCLUSION

Kuber packages the full stack needed to build a CHT surrogate — a reproducible data engine, a geometry-general surface-conditioned transformer, and an honest evaluation harness — and demonstrates it on electronics cooling, reaching state of the art on a public benchmark without domain adaptation and spanning two device classes from one model. The same interfaces target the broader class of coupled fluid-heat problems. Calibrated uncertainty, CAD connectors, and agentic geometry optimization are natural next steps; code, data, and an interactive viewer are released to support them.

## References

- [1] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, “PointNet: Deep learning on point sets for 3D classification and segmentation,” in *Proc. IEEE CVPR*, 2017.
- [2] P. Setinek *et al.*, “SIMSHIFT: A benchmark for distribution shift in engineering simulation,” preprint, 2025.
- [3] H. Wu, H. Luo, H. Wang, J. Wang, and M. Long, “Transolver: A fast transformer solver for PDEs on general geometries,” in *Proc. ICML*, 2024.
- [4] B. Alkin, A. Fürst, S. Schmid, L. Gruber, M. Holzleitner, and J. Brandstetter, “Universal physics transformers,” in *Proc. NeurIPS*, 2024.
- [5] B. Alkin *et al.*, “Anchored/branched universal physics transformers,” preprint, 2025.
- [6] Z. Li *et al.*, “Fourier neural operator for parametric partial differential equations,” in *Proc. ICLR*, 2021.
- [7] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, “Learning nonlinear operators via DeepONet,” *Nature Machine Intelligence*, 2021.
- [8] Z. Li *et al.*, “Geometry-informed neural operator for large-scale 3D PDEs,” in *Proc. NeurIPS*, 2023.
- [9] P. Lippe, B. Veeling, P. Perdikaris, R. Turner, and J. Brandstetter, “PDE-Refiner: Achieving accurate long rollouts with neural PDE solvers,” in *Proc. NeurIPS*, 2023.
- [10] NVIDIA, “PhysicsNeMo: A framework for physics-ML (GeoTransolver),” software, Apache-2.0.
- [11] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.
- [12] H. G. Weller, G. Tabor, H. Jasak, and C. Fureby, “A tensorial approach to computational continuum mechanics using object-oriented techniques,” *Computers in Physics*, vol. 12, no. 6, 1998.